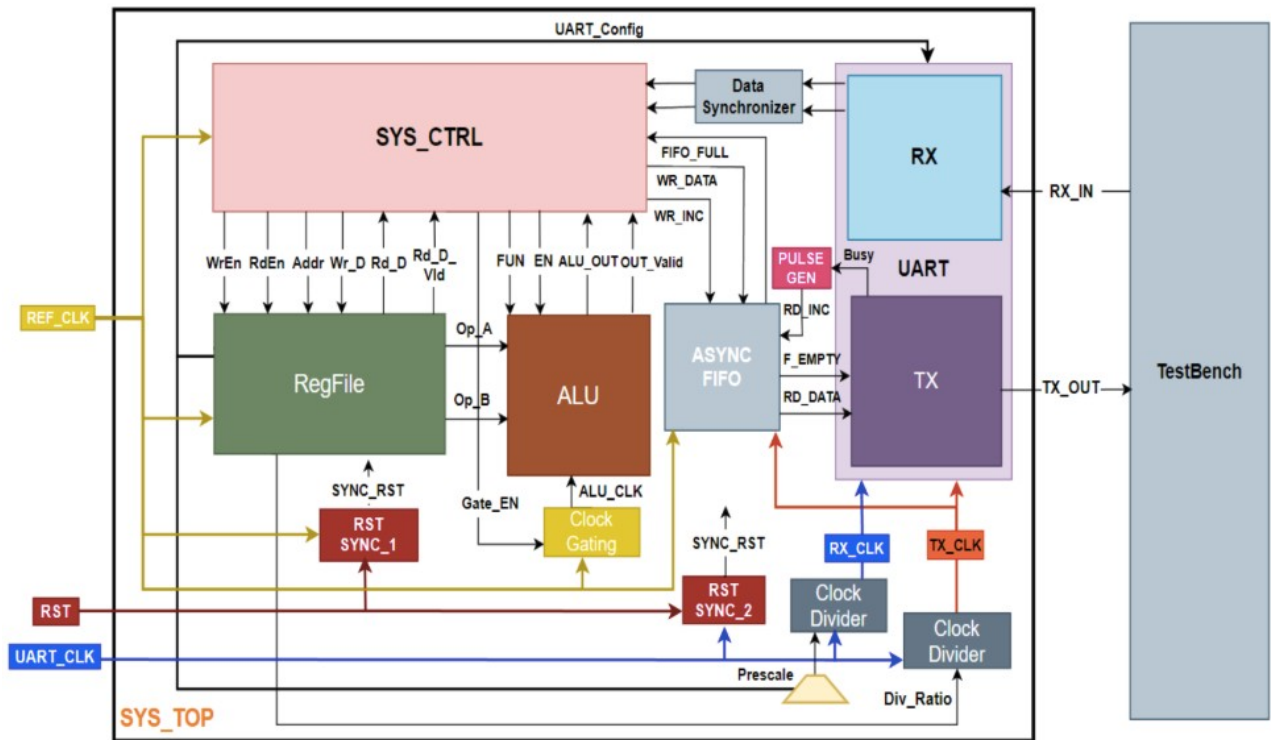


Final System Design & Integration & Functional Verification

Zeyad Khaled Abd Elhamid El Razek



First lets go through the most important Tasks in the TB

```
task initialize;

    REF_CLK=0;
    UART_CLK=0;
    RX_IN=1;
    RST=0;
    scan_clk=0; scan_rst=0; test_mode=0; SE=0; SI='0
repeat (2) @(negedge UART_CLK);
    RST=1;
repeat (1) @(negedge UART_CLK);

endtask
```

```
task automatic send_frame (
    input logic [WIDTH-1:0] Data,
    input logic par_bit
);
begin
    RX_IN=0;
    repeat (rx_prescale) @(negedge DUT.rx_clk);

    for(int i=0 ; i<WIDTH ; i++) begin
        RX_IN=Data[i];
        repeat (rx_prescale) @(negedge DUT.rx_clk);
    end

    RX_IN=par_bit;
    repeat (rx_prescale) @(negedge DUT.rx_clk);

    RX_IN=1;
    repeat (rx_prescale) @(negedge DUT.rx_clk);
end
endtask
```

The Send Frame Sends data in serially acting like the sending tx for the RX to receive any frame must start with 0 to be a valid frame , followed by the data or command , followed by the par_bit then stop bit

```

task modify_configures (
    input [WIDTH-1:0] CMD,
    input [RF_ADDR_WIDTH-1:0] ADDR,
    input [WIDTH-1:0] config_data
);
    begin
        send_frame(CMD, ^CMD);
        send_frame(ADDR, ^ADDR);
        send_frame(config_data, ^config_data);

        if(ADDR=='h02) begin
            rx_prescale=config_data[WIDTH-1:2];
        end

        repeat (rx_prescale) @(negedge DUT.rx_clk);
    end
endtask

```

Sends frames to modify the REG signals at the start of the simulation

```

task Wr_Data (
    input [WIDTH-1:0] CMD,
    input [RF_ADDR_WIDTH-1:0] ADDR,
    input [WIDTH-1:0] wr_data
);
    begin
        send_frame(CMD, ^CMD);
        send_frame(ADDR, ^ADDR);
        send_frame(wr_data, ^wr_data);
        repeat (rx_prescale) @(negedge DUT.rx_clk);
    end
endtask

task Rd_Data (
    input [WIDTH-1:0] CMD,
    input [RF_ADDR_WIDTH-1:0] ADDR
);
    begin
        send_frame(CMD, ^CMD);
        send_frame(ADDR, ^ADDR);
        repeat (rx_prescale) @(negedge DUT.rx_clk);
    end
endtask

```

```

task ALU_W_OP (
    input [WIDTH-1:0] CMD,
    input [WIDTH-1:0] OP_A,
    input [WIDTH-1:0] OP_B,
    input [FUN_WIDTH-1:0] ALU_FUNC
);
    begin
        send_frame(CMD, ^CMD);
        send_frame(OP_A, ^OP_A);
        send_frame(OP_B, ^OP_B);
        send_frame(ALU_FUNC, ^ALU_FUNC);
        repeat (rx_prescale) @(negedge DUT.rx_clk);
    end
endtask

task ALU_W_NOP (
    input [WIDTH-1:0] CMD,
    input [FUN_WIDTH-1:0] ALU_FUNC
);
    begin
        send_frame(CMD, ^CMD);
        send_frame(ALU_FUNC, ^ALU_FUNC);
        repeat (rx_prescale) @(negedge DUT.rx_clk);
    end
endtask

```

TB TEST CASES :

initial begin

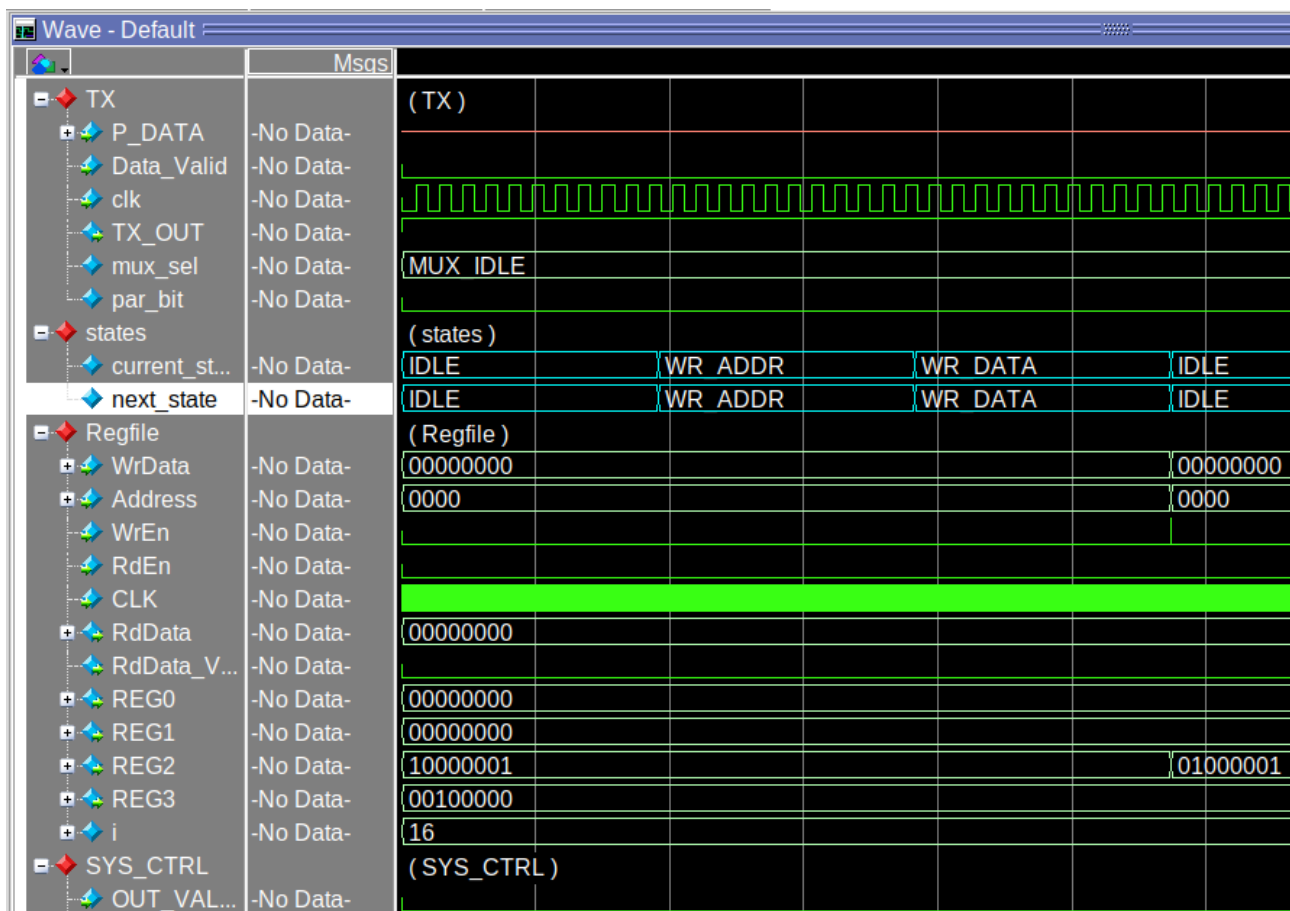
 initialize;

 repeat (10) @(negedge DUT.rx_clk);

 modify_configures('hAA','h02','h41');

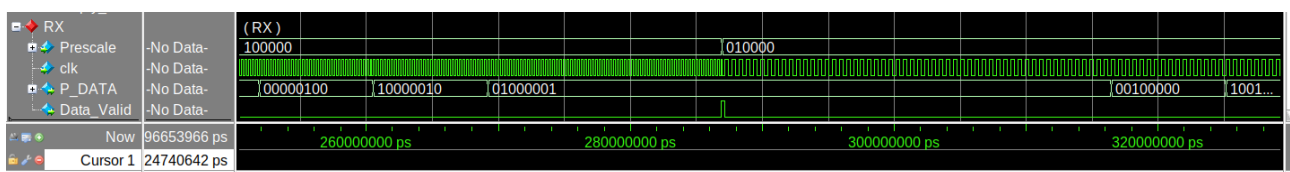
 modify_configures('hAA','h03','h20');

1st



Modifying REG2 by using WR_CMD

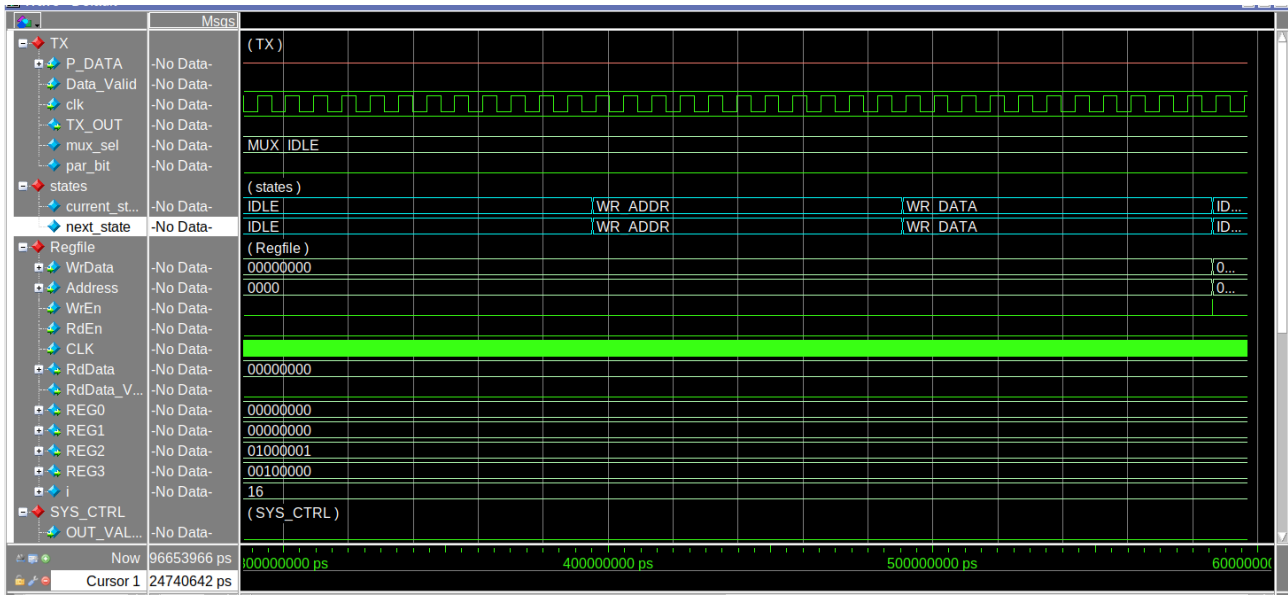
originally RX Prescale = 32 the highest clk freq we will modify it to 16



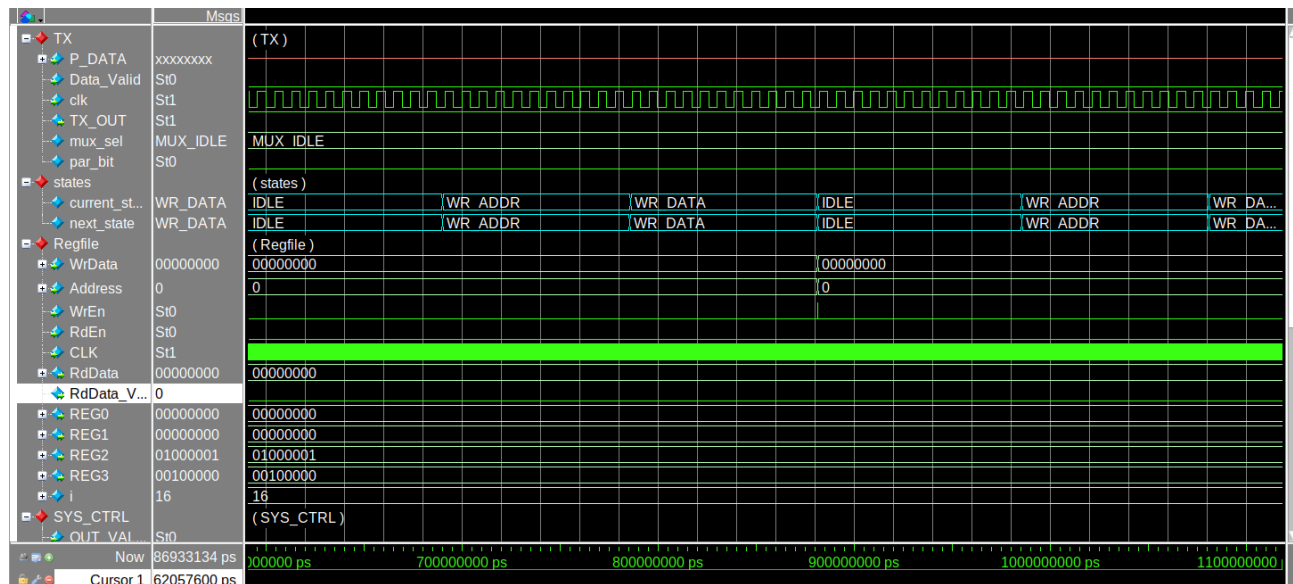
RESULT :

RX CLK Freq is /2 or 2* period

Case 2 : reg3 is kept at its 32 value



WR_CMD, ADDR, DATA

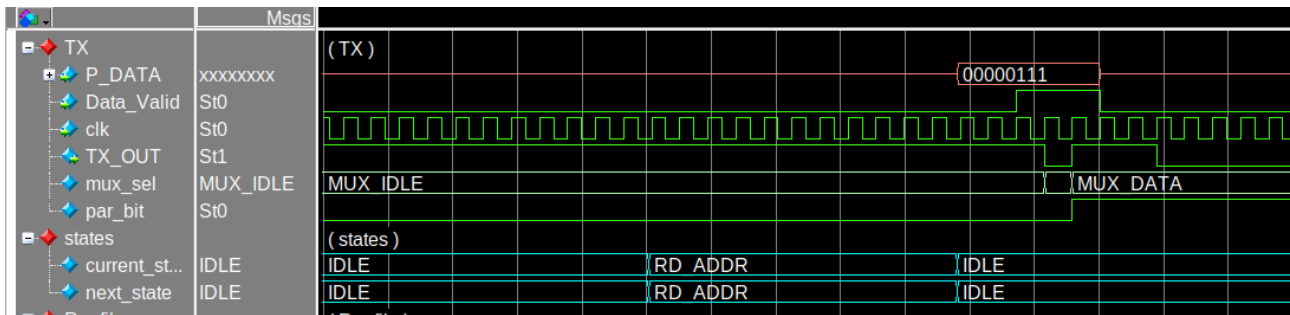
[illegible][illegible]

Now Do Rd_Data to check they were written successfully

Case 4 : Rd Data

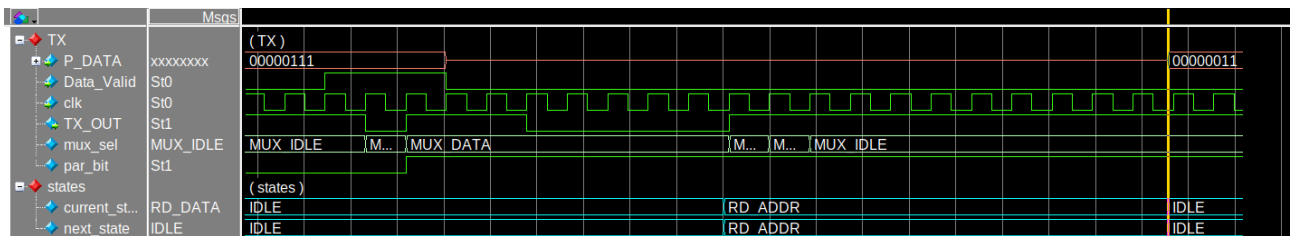
```
Rd_Data('hBB','h05);
```

```
Rd_Data('hBB','h09);
```



States were successful

The TX P_data Sucessfully receieved the Rd_Data from the Regfile

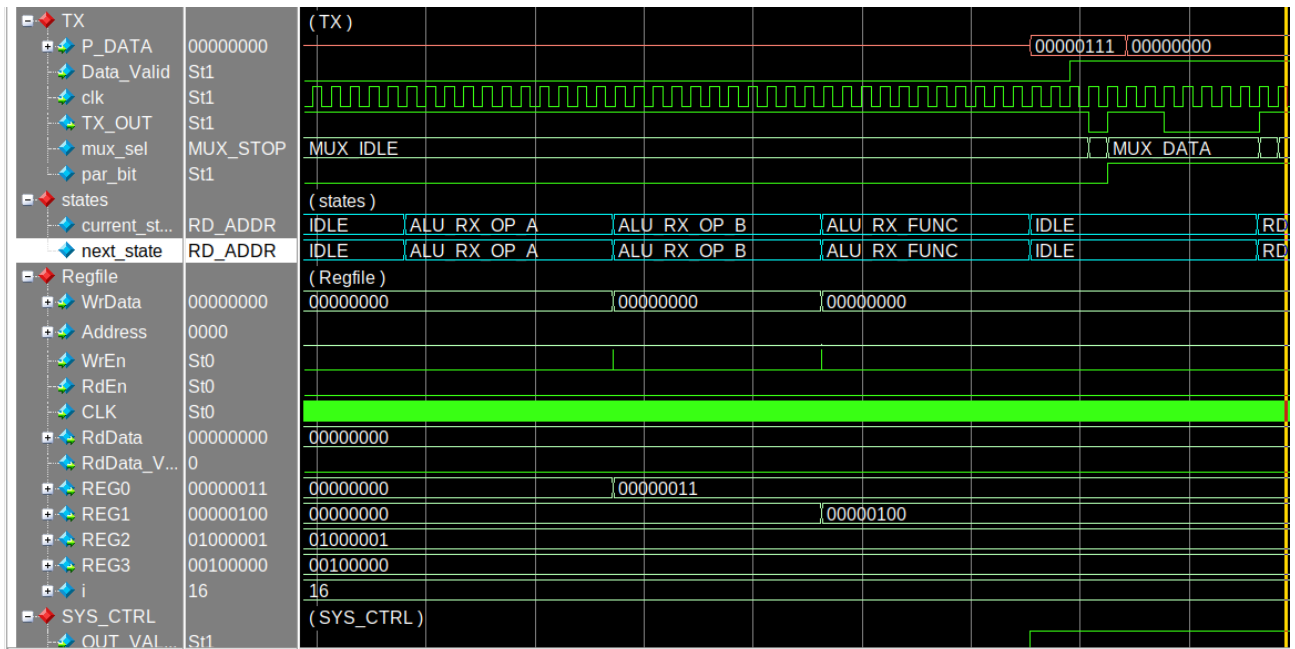


successfully captured 011 data

Case 5 : ALU WITH OPERANDS

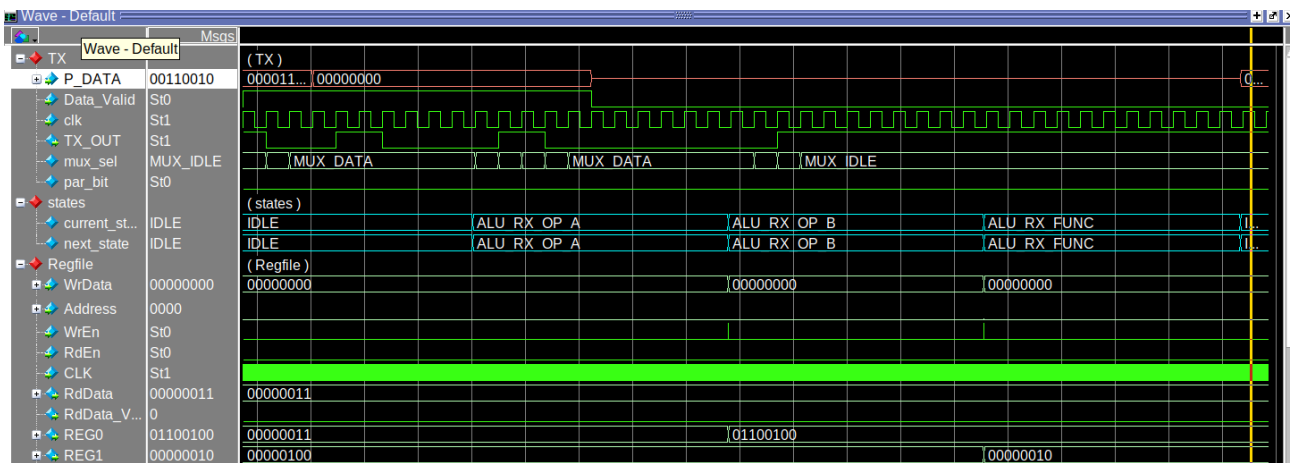
ALU_W_OP('hCC,3,4,'h0000);

1.



first REG 0 = 3 , REG1 = 4 , ALU_FUN = 0000

P_DATA Sucesfully recieved frame 1 Result 7 and frame 2 result = 0

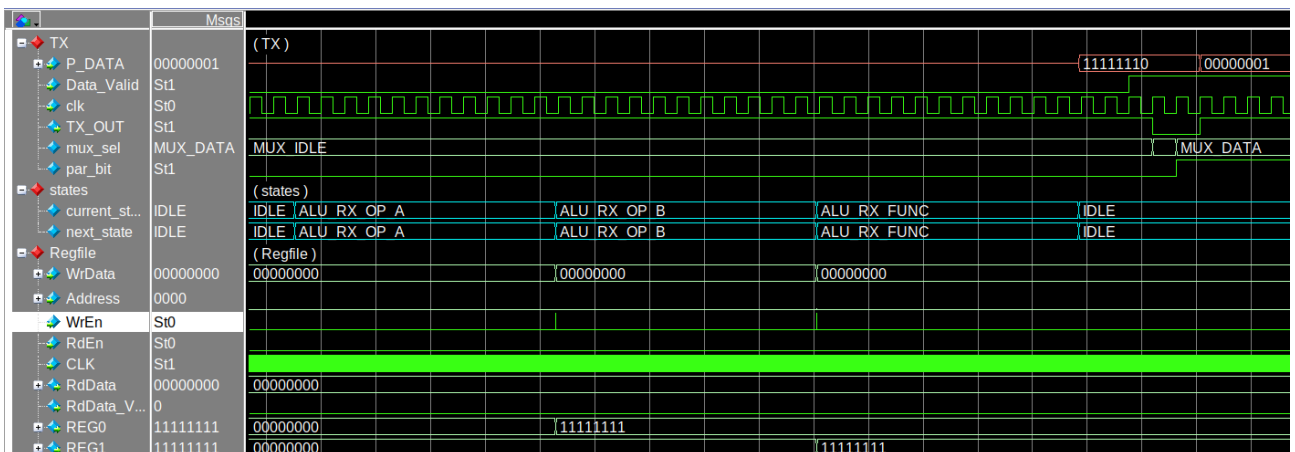


ALU_W_OP('hCC,100,2,'b0011);

A=100 , B= 2 , ALU_FUNC =Divide

P_DATA = 00110010 → 50

ALU_W_OP('hCC,8'b11111111,8'b11111111,'h0000);



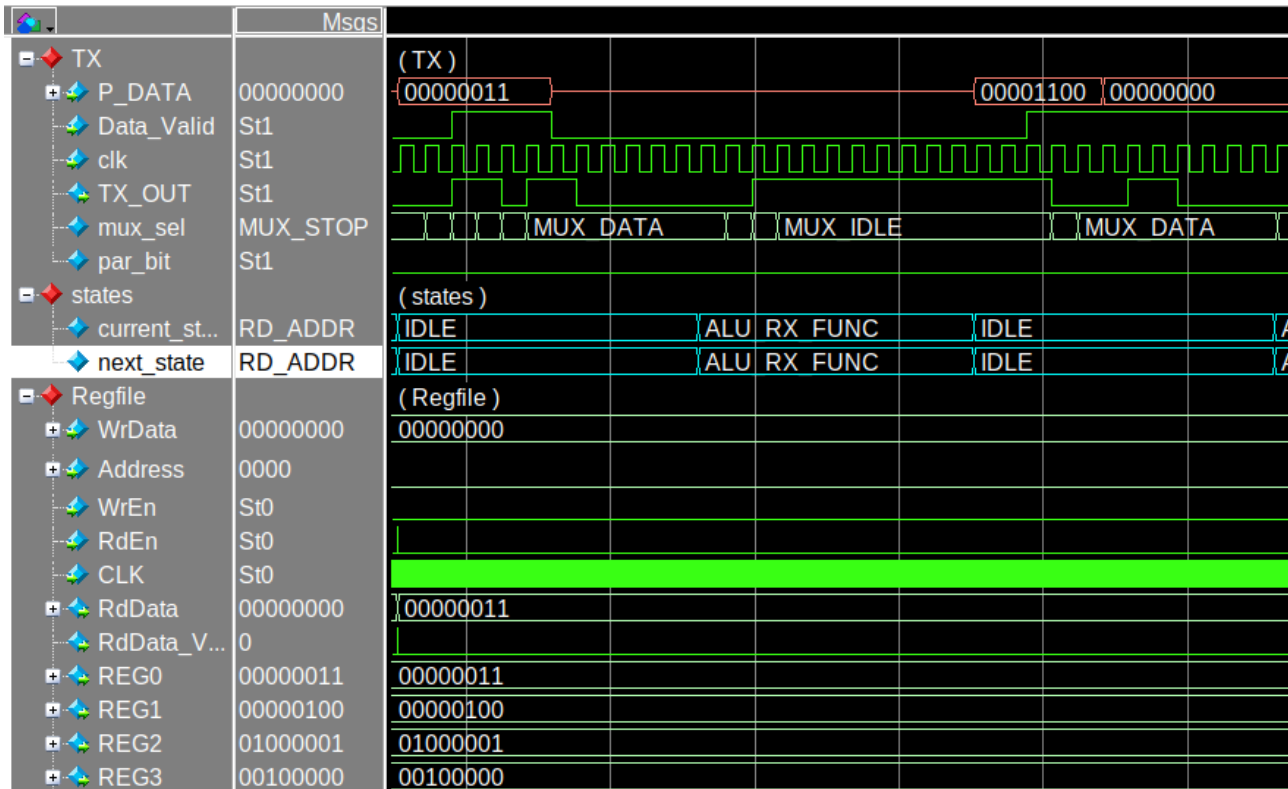
Multiplying $256 * 256 = 65025$

The output is sent on two frames

P_DATA = 11111110_00000001 → 65025

Correct

Test Case 6 No Operands :



ALU_W_NOP('hDD,'b0010);

now we test No operands ALU_ Operation for the same A and B Values;

ALU_FUNC = 0010 → Multiply

P_DATA Successfully captures $3 * 4 = 1100 \rightarrow 12$